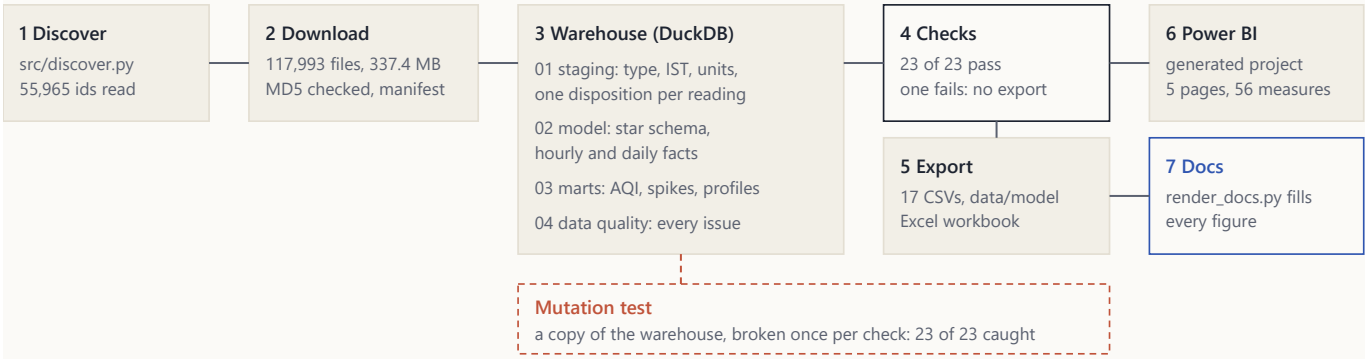
REVIEWER'S GUIDE

How it works

For a reviewer with fifteen minutes: where everything lives, how to rebuild it, where each number comes from, how it is proved, and the methods most worth challenging.

India Air Quality · Python, DuckDB, SQL, Excel, Power BI · 107 stations, 2019-01-01 to 2026-09-30 · read with the case study

1. The flow in one picture



Proof: 23 of 23 checks gate the export · 23 of 23 breakages caught · 18 of 18 output files identical on rebuild · 423 of 423 report figures match SQL

> **The fifteen-minute review.** The raw files and the warehouse are not in git (they are rebuilt from the archive), so a short review reads; a full proof rebuilds.
>
> 1. Read `sql/checks.sql` next to `src/mutation_test.py` : one named check per rule, one deliberate break per check.
> 2. Read the CPCB sub-index in `sql/03_marts.sql` against the breakpoint table in section 7, and the second, independent version inside `checks.sql` .
> 3. Read the spike rule in `src/config.py` and its three marts at the end of `03_marts.sql` .
> 4. Open `data/pipeline.log` , `mutation.log` and the two `.sha` files after a build, or rebuild (section 3).

2. Repo map

PATH	WHAT IT DOES
`src/config.py`	Settings that change the dataset: window, cities, budget (350 MB), station and spike rules
`src/discover.py`	Finds the stations in the archive itself, with no API key; resumable
`src/download.py`	Budget-capped, resumable download; MD5 per file; a manifest and its committed summary
`src/pipeline.py`	Rebuilds `data/aq.duckdb` from scratch, runs the checks, exports only if all pass
`sql/01_staging.sql`	Typed readings on IST in canonical units, one disposition each (`stg_readings`)
`sql/02_model.sql`	Dimensions for city, station, pollutant and date; hourly and daily measurement facts

PATH	WHAT IT DOES
`sql/03_marts.sql`	AQI, months, seasons, exceedance, spikes, Oct-Nov, hourly profiles, completeness
`sql/04_dq.sql`	Every data issue per city, station and pollutant, with the action taken
`sql/checks.sql`	23 of 23 checks; each returns the rows breaking its rule, so a pass is an empty result
`src/mutation_test.py`	Breaks a copy of the warehouse once per check; each check must then fail
`src/findings.py` , `excel.py`	`docs/findings.md` ; the workbook, 12 sheets and 9 Excel Tables
`powerbi/build_pbip.py`	Writes the whole Power BI project (TMDL model, PBIR report) from the warehouse
`powerbi/check_*.py`	`check_report.py` : every report figure against SQL; `check_fit.py` : no text clipped
`powerbi/*.ps1`	Open in its own Desktop window, query, capture, trim, validate TMDL, close that window
`reference/`	Every station with its inclusion decision and reason, AQI breakpoints, pollutants
`docs/`	`render_docs.py` fills every figure; `charts.py` draws the SVGs; `build_pdf.ps1` prints

3. Rebuilding it

```
python -m venv .venv
.venv/Scripts/pip install -r requirements.txt
.venv/Scripts/python src/download.py --plan-only  # list and size; download nothing
.venv/Scripts/python src/download.py              # 337.4 MB to data/raw; resumable
.venv/Scripts/python src/pipeline.py              # warehouse, checks, exports
.venv/Scripts/python src/mutation_test.py         # breaks a copy once per check
.venv/Scripts/python src/findings.py              # docs/findings.md
.venv/Scripts/python powerbi/build_pbip.py        # DataFolder = this checkout
.venv/Scripts/python docs/render_docs.py          # stops on a stale figure
# then open powerbi/AirQuality.pbip in Power BI Desktop and choose Refresh
```

Needs Python 3.12+, about 12 GB free and, for the report, Power BI Desktop; discovery is optional, as `reference/stations.csv` is committed. Times are from the logs, none estimated: download 37 min for 107,705 files (289.2 MB); 2.2 min for 10,288 files (48.2 MB) (`data/manifest.csv`); mutation test 2.5 min (`data/mutation.log`); the SQL stages in the two builds compared for determinism:

STAGE	BUILD A	BUILD B	LOGGED IN
`01_staging.sql` , 56,030,781 raw rows	22 min	20 min	`data/build_a.log` and `build_b.log` ; `pipeline.log` is build A
`02_model.sql`	2.3 min	51 s	
`03_marts.sql`	47 s	21 s	
`04_dq.sql`	28 s	11 s	
All four SQL stages	26 min	22 min	

What a clean run prints: `download.py --plan-only` gives `plan: 117993 files, 337.4 MB from 122 locations` ; `pipeline.py` gives one timed `built` line per SQL file, 23 lines starting with `pass` , `exported 17 tables to data/model` and the workbook path; `mutation_test.py` ends with `23 of 23 mutations caught` and its time. Any `FAIL` or `MISSED` line ends in a non-zero exit.

4. Where each number comes from

Every headline figure is one query on one table, and one check recomputes that table another way. `render_docs.py` runs the same queries for the README, the case study and this guide, and stops if a token is unfilled, if the exports differ from the logged build's hashes, or if the warehouse's spike rule differs from `src/config.py`. `src/findings.py` recomputes the findings for `docs/findings.md` and the workbook; `check_report.py` compares the report with SQL (section 6).

FIGURE	VALUE	TABLE	RECOMPUTED BY THE CHECK
Stations (OpenAQ ids)	107 (122)	`dim_station`	each location in one station, one city
Raw rows	56,030,781	`raw_file_rows`	every raw row staged or out of scope, once
Valid readings	36,075,426	`stg_readings`	one disposition each; unique, in range, on IST
Readings set aside	1,089,800	`mart_dq_summary`	counts reconcile with the staged readings
"ppb" readings relabelled	12,171,979	`mart_dq_issues`	the unit evidence, retested
City-days with an AQI	7,083	`mart_city_daily_aqi`	rounded mean of the station AQIs
Worst city-month	Delhi Nov 2021, 401	`mart_city_month`	recount from the daily tables
Days over the PM2.5 limit	Delhi 56.5%, most	`mart_exceedance_city_year`	recount against the NAAQS limits
Short spikes	79	`mart_spike_episodes`	spike days, runs and peaks rebuilt
Delhi Oct-Nov vs September	3.3x to 5.3x	`mart_octnov_season`	recount from city daily PM2.5
Worst hour for PM2.5	22:00 IST	`mart_profile_hourly`	hours add up to the city hourly table
Station-hours with a reading	28%	`mart_station_completeness`	valid hours recounted, 0 to 100%

5. How it is proved

Checks gate the export

Each check in `sql/checks.sql` is a named query that returns the rows breaking its rule. `pipeline.py` runs all of them after the four SQL stages; if any returns a row it prints it, exports nothing and exits with an error. The checks recompute rather than restate; the PM2.5 and PM10 sub-indices are recomputed by a second, hand-written implementation of the CPCB bands. All 23 of 23, as named in the file, with the result from `data/mutation.log`:

#	CHECK	MUTATION
1	every raw row of an included station is staged or out of scope, once	caught
2	every staged reading has exactly one known disposition	caught
3	valid readings are on IST, in the window, in range and unique per station, pollutant and time	caught
4	"ppb" gas labels are CPCB units, not ppb (CO median under 20, NO2 level matches the same stations before)	caught
5	hourly values equal the mean of their valid readings, with the reading count	caught

#	CHECK	MUTATION
6	no hour or day is invented: every fact row has valid readings behind it	caught
7	daily 24-hour means and valid-hour counts recompute from the hourly fact	caught
8	a day is complete exactly when it has 16 or more valid hours, and only complete days get a sub-index	caught
9	PM2.5 and PM10 sub-indices match an independent implementation of the CPCB breakpoints	caught
10	every sub-index is between 0 and 500 and rises with concentration	caught
11	station AQI is the highest sub-index, from three or more pollutants including PM	caught
12	city AQI is the rounded mean of the station AQIs, with the right station count and category	caught
13	city-month figures recount from the city daily tables	caught
14	exceedance days recount from daily values against the NAAQS limits	caught
15	spike days: the reference is the median of the surrounding measured days, and a spike day is at least the set ratio above it	caught
16	spike episodes are whole runs of 1 to the set maximum of spike days, with evaluated ordinary days on both sides	caught
17	spike episode peaks, ratios and station counts recompute from the spike days and stations	caught
18	October-November and September levels recount from the city daily PM2.5	caught
19	profile hours add up to the city hourly table	caught
20	completeness counts valid hours correctly and stays between 0 and 100 percent	caught
21	data-quality counts reconcile with the staged readings	caught
22	every included location belongs to one station and every station to one of the six cities	caught
23	the calendar is continuous over the window	caught

Mutation test

`mutation_test.py` holds one break per check and asserts the two lists match, so a check without a break fails the test. For each, it copies the warehouse to `data/mutation.duckdb`, confirms the check passes, applies the break (a deleted reading, an invented hour, a nudged sub-index, a station moved to another city...) and requires the check to return rows. The real warehouse is never touched; the run took 2.5 min.

Determinism

- **Exact numbers.** Readings are stored as `DECIMAL(18, 6)`, hourly and daily values as rounded decimals and the breakpoints as `DECIMAL(10, 3)`. Sums are exact, so means do not depend on the order DuckDB's threads add them; no thread pinning is needed.
- **Fixed order.** Every table is exported with `ORDER BY ALL`; the workbook uses fixed document dates and sorted rows; report object ids are hashes of their names, not random.
- **Proof.** `data/build_a.sha` and `build_b.sha` hold the SHA-256 of the 18 outputs (17 CSVs and the workbook) after two builds from the raw files: 18 of 18 identical. `render_docs.py` re-hashes the current files against build B on every run.

To repeat it (Git Bash), hash the outputs after each of two builds from the raw files and compare:

```
sha256sum -b data/model/*.csv excel/india-air-quality.xlsx > data/build_b.sha
diff data/build_a.sha data/build_b.sha    # no output: identical
```

6. The report: generated, then reconciled

STEP	WHAT IT DOES	RESULT
`build_pbip.py`	Writes the TMDL model and PBIR report. Column types come from the warehouse, colours and heatmap geometry from the data; every answer sentence is a measure; `DataFolder` points at this checkout	14 tables, 20 relationships, 56 measures, 5 pages
`validate_tmdl.ps1`	Parses the model with Power BI Desktop's own serializer, without opening Desktop	parses cleanly
`check_report.py`	Finds measures in error and evaluates each; then evaluates every figure the pages show under default slicers (cards, heatmap cells, spike table, issue log, whole answer sentences) against independent SQL	56 of 56 measures; 423 of 423 figures
`check_fit.py`	Measures every title, label, cell and sentence in the report's fonts against its box, with a 6% margin; answers and cards at their widest under any slicer	305 of 305 lines, 36 of 36 wrapped, 14 of 14 grids
`capture_pages.ps1`	Renders each page in its own Desktop window and trims it to the canvas by the rail colour	5 page images

7. The AQI method (CPCB 2014)

CPCB, *National Air Quality Index*, CUPS/82/2014-15. Linear interpolation inside the band that holds the concentration, the published integer bands treated as continuous; at or above the top of Severe, 500:

```
I = I_lo + (I_hi − I_lo) / (B_hi − B_lo) * (C − B_lo), rounded to a whole number
```

Pollutant, unit, averaging	Good 0-50	Satisfactory 50-100	Moderate 100-200	Poor 200-300	Very Poor 300-400	Severe 400-500
PM2.5 µg/m³, 24-hour mean	0-30	30-60	60-90	90-120	120-250	250-380
PM10 µg/m³, 24-hour mean	0-50	50-100	100-250	250-350	350-430	430-510
NO2 µg/m³, 24-hour mean	0-40	40-80	80-180	180-280	280-400	400-520
O3 µg/m³, maximum 8-hour mean	0-50	50-100	100-168	168-208	208-748	748-1,288
CO mg/m³, maximum 8-hour mean	0-1	1-2	2-10	10-17	17-34	34-51
SO2 µg/m³, 24-hour mean	0-40	40-80	80-380	380-800	800-1,600	1,600-2,400

- **Averaging.** 24-hour mean for PM2.5, PM10, NO2 and SO2; for O3 and CO the highest 8-hour rolling mean whose window holds 6+ valid hours. A day needs 16 valid hours, or it gets no daily value and no sub-index.
- **Station AQI** is the highest sub-index, published only with 3+ pollutants on complete days, one of them PM2.5 or PM10; that pollutant is the prominent one. **City AQI** is the rounded mean of that day's station AQIs, as in CPCB's bulletin.
- **Severe is open-ended** in CPCB's table; here it closes at its lower bound plus the width of the Very Poor band (the last column). NH3 and lead are not in the archive.
- **Before any averaging,** readings above a plausibility ceiling are set aside as faults: PM2.5 1,000, PM10 2,000, NO2 1,000, O3 1,000, SO2 2,000 µg/m³; CO 50 mg/m³. Exceedance uses the NAAQS: PM2.5 60, PM10 100, NO2 80, SO2 80 µg/m³ (24-hour); O3 100 µg/m³ (8-hour); CO 2 mg/m³ (8-hour).

8. The spike rule

Found from the data alone: no event calendar, no labels, no causes. The settings in `src/config.py` are written into the warehouse as `ref_spike_rule`, so the marts, the checks and the report read the same values.

SETTING	VALUE	MEANING
`SPIKE_MIN_RATIO`	2.0	a spike day's city PM2.5 is at least this multiple of its reference
`SPIKE_GUARD_DAYS`	3	days either side left out of the reference, so an episode never sets its own
`SPIKE_WINDOW_DAYS`	14	the reference is the median of days 4 to 14 before and after
`SPIKE_MIN_DAYS_EACH_SIDE`	6	measured days needed on each side (of 11) for a day to be evaluated
`SPIKE_MAX_DAYS`	3	longest run of spike days that counts as a short spike

1. `mart_spike_days` and `mart_spike_runs`: 144 of 5,724 evaluated city-days are spike days (2.5%), in 100 runs: 79 short spikes, 3 longer than 3 days and 18 next to a day without a reading or reference (length unknown). Only short spikes count; the rest keep their reason.
2. `mart_spike_episodes`: each short spike's peak, reference and ratio, and the stations at 2.0x the city reference on the peak day. 22 are carried by one station of several; 2 rest on one reporting station.

9. Choices worth challenging

Set aside, never fill. Nothing is interpolated or estimated: 27 months stay empty, and every value shown was measured.

"ppb" relabelled, not converted. As true ppb, NO2 would jump at the gap and CO would fall to near zero. The evidence is a check: CO "ppb" median under 20; NO2 within 0.67 to 1.5 times its earlier level.

Conflicting duplicates: all copies set aside. Where two ids of one station disagree at one timestamp there is no basis to choose, so 475,812 readings go; averaging them would invent a value neither instrument reported.

A round spike threshold. Double the surrounding level, not a fitted value; change it in `config.py` and everything follows.

10. Known limits

- No Indian data in the archive from 31 Oct 2022 to 19 Feb 2025; no weather or emissions data, so no causes.
- Station sets grow and city AQI is a mean of stations, so city figures across years are not like-for-like.
- Raw files and warehouse are not in git; no script writes the `.sha` files; discovery and the report checks are untimed.
- Power BI has no free public link (the report ships as a project with images); the workbook has Excel Tables, not PivotTables.

Every figure in this guide is filled from the warehouse, the build logs, the SQL and `src/config.py` by `docs/render_docs.py`. Data: CPCB and state boards, via OpenAQ.